

Soulstream — Design Documents

2026-07-11

Contents

Soulstream — Design Docs	2
Reading order	2
Decision log (vs. the earlier Impire-era draft)	2
Build order	4
Status	5
The gap	5
Principles	5
What v1 is	6
Non-goals for v1	6
Realms	6
Streams	7
SOULSTREAM	7
Object store: soulstream-objects	7
KV: soulstream-personas	7
Subject taxonomy	7
The operation record	8
The canonical op record	9
Size discipline	9
What the substrate does <i>not</i> provide	10
Naming	10
The registry	10
Credentials	10
Delegation	11
Roles	11
Mentions and attention	12
Starting a topic	12
Lifecycle	13
Operation vocabulary	13
Comments, anchors, mentions	13
Attachments	14
Concurrency: eg-walker over JetStream	14
Baselines	14
The single-message invariant, and what happens when state outgrows it	15
The steward	15
Other op-log surfaces	16
Layer 0 — the wire spec	16
Layer 1 — the topic library	16
Layer 2 — adapters	16
The MCP adapter (agents' door)	17
The WebSocket gateway (browsers' door)	17
Bridges (later, but shaped now)	17

Search is a participant concern	17
Presence and attention signals	18
What a minimal deployment looks like	18
Open questions (tracked, not blocking)	18
Threat model — what sealing does and does not protect	18
Persona keys	19
Epoch keys	19
Sealed operations	20
Announcements, baselines, attachments, mentions	20
Interaction with the rest of the design	21
Cost, stated plainly	21
Participant-local memory	21
The query convention	21
The epistemics: signatures make anyone a witness	22
Archivists are an optimisation, not a requirement	23
Service announcements	23
Sealed topics and memory	23
What the substrate commits to	24
The MVP criterion	24
Why deferring is safe here	24
One-way doors (sequencing constraints, not features)	24
MVP — in scope, minimal slice per capability	24
Day-2 — next, in rough order	26
Later	26
The work ladder	26

Soulstream — Design Docs

A headless collaboration substrate on NATS where humans and AI personas participate as peers. Not a startup; infrastructure built for its own sake.

Reading order

1. [00-vision.md](#) — what it is, the five principles, non-goals.
2. [01-substrate.md](#) — realms, streams, subject taxonomy, the operation record (headers on the wire, pure-data payloads).
3. [02-personas.md](#) — the identity model: registry, credentials, delegation, mentions.
4. [03-topics.md](#) — topics as op-logs: vocabulary, eg-walker merge, baselines, the steward.
5. [04-protocol.md](#) — the three connection layers, adapters (MCP, WebSocket), deployment shape, open questions.
6. [05-sealed-topics.md](#) — E2E-encrypted topics: threat model, epoch keys, sealed ops, what deliberately stays visible.
7. [06-memory.md](#) — memory and collective search: participant-local indexes, scatter-gather queries, signed ops as self-authenticating evidence, optional archivists.

Decision log (vs. the earlier Impire-era draft)

Decision	Was	Now	Why
Standing	One stream inside Impire	The whole platform	Fresh start; all Impire dependencies (System, Guards, storage service, cockpit) removed or absorbed.
Vocabulary	imps / keepers / tenant	personas / realm	Humans and AIs share one noun by design.
Curation	Privileged “System” component	Steward: an ordinary agent persona	No privileged plumbing; curation is replaceable and opt-out.
State vs ops	Stream MaxAge + 3 compensating cleanup mechanisms; snapshots as external state	Ops stream with a moving baseline : baseline is always the topic’s first message, from birth; rollup = old baseline + ops → new baseline. Always one message: inline ≤128 KB, chunk manifest beyond (chunks put first, single-message publish is the atomic commit). No MaxAge. JetStream object store bucket per realm	The Soulstream carries operations, not state (Daan); the baseline is the zero-point ops are relative to, not state pollution. Single-message rule is the crash-safety line: chunking a baseline across the stream can’t survive rollup + a mid-publish crash.
Blob storage	External storage platform service		Single-dependency deployment; swappable later behind the same reference convention.
Delegation	(unspecified)	Scoped credentials only; no on_behalf_of field	Refuses attribution laundering: it’s either your credential or another persona you operate.
Identity kind	Structural (imps vs keepers)	kind is presentation metadata; behaviour may never branch on it	The peer principle, made testable.
Sandboxes	(aspiration)	Explicit non-goal for v1; op-log hooks reserved	Different problem class (filesystems/processes, not messages); design against a real substrate later.

Decision	Was	Now	Why
Confidentiality	(unaddressed)	Sealed topics: E2EE with wrapped epoch keys, operator excluded; MLS as upgrade path	Threat model includes the operator; permission-sealing alone only excludes peers. Sealed = enforced membership, blind steward, metadata still visible.
Search	(open question)	Collective memory (doc 06): participant-local indexes + scatter-gather queries; answers graded by citation (live / signed exhibit / unsigned exhibit / uncited=gossip)	No privileged plumbing; the realm's memory is the union of what participants bothered to remember. Compaction bounds retroactive indexing — index from when you start caring.
Wire format	Envelope JSON wrapping every payload	Record-in-headers: Nats-Msg-Id is the op ID, Ss-* headers carry author/parents/type/sig; payload is pure data. Canonical op record (JCS JSON) defined once for signing, exhibits, and sealed inner ops	A NATS message is already an envelope (Daan). Kills the id/Nats-Msg-Id duplication; sealed payloads become raw ciphertext; interchange keeps a canonical form because signatures and exhibits need one.
Provenance	Attribution = transport only	Optional Ed25519 sig per op: live attribution stays credential-based, durable attribution is signature-based; any kept signed op is self-authenticating evidence	Anyone can be a witness, so archivists (historian/librarian) are a coverage optimisation added when needed — not a trust anchor. Reputation stays a social fact; no reputation mechanism in the substrate.

Build order

[ROADMAP.md](#) — the MVP cut (scenario-driven, per-capability minimal slices), the one-way doors that constrain sequencing, and day-2/later staging. The design docs describe the whole system; the roadmap decides what exists when.

Status

v1 design, 2026-07-10. Open questions are tracked at the end of [04-protocol.md](#). # Soulstream
A headless collaboration substrate where humans and AI personas work as peers.

The gap

Collaboration platforms — Notion, Google Workspace, Slack — were built for humans, then had AI bolted on as a feature: an assistant in a sidebar, a bot with a special API, a “copilot” that lives outside the document model. The AI is always a second-class citizen with a different door into the building.

Soulstream inverts this. It is a substrate, not a product: a set of NATS streams, subject conventions, and a client library. Every participant — human or AI — is a **persona** that connects the same way, speaks the same protocol, and appears in the same attribution model. There is no bot API and no human API. There is one protocol.

Work happens in **topics**: focused, multi-party conversations that carry turns, comments, attachments, edits, and angles as operations on a shared log. Topics are where personas meet; everything else is convention layered on top.

Principles

One protocol, no second door. A persona is a persona. Humans and AI agents hold the same kind of credentials, publish the same operation record, and are addressed the same way. Whether a persona is backed by a person at a keyboard or a model in a loop is *metadata for presentation*, never *mechanism*. Anything that only works for humans, or only works for agents, is a design smell.

Protocol symmetry, attention asymmetry. Equality at the protocol layer does not mean equality of capacity. An AI persona can subscribe to every topic in a realm and read every operation; a human cannot. The scarce resource in the system is human attention, and the substrate must budget for it explicitly: mention routing, topic projections, digests, and curation are first-class concerns, not UI afterthoughts. A substrate designed only for the fast reader drowns the slow one.

Headless means the substrate is the product. There is no canonical UI. The platform is: a NATS deployment, the subject and stream conventions, the operation vocabulary, and a client library that implements them. Web apps, TUIs, MCP servers, and autonomous agents are all *heads* — clients of the same body. If a capability only exists in a head, it isn’t part of the platform.

No privileged plumbing. Above NATS itself, there are no special services. Curation, digesting, archiving — jobs a platform would normally hide in backend services — are performed by personas holding ordinary credentials, publishing ordinary operations. A steward persona that flags duplicate topics uses the same protocol as everyone else. This keeps the substrate honest (if the protocol can’t support the steward, it can’t support your agents either) and makes every platform behaviour replaceable, inspectable, and opt-out.

Convention over enforcement. The substrate enforces little: subject permissions decide who can publish where, and that is nearly the whole security model. Everything above that — operation vocabularies, topic etiquette, lifecycle transitions, roles — is convention that participants and libraries agree on. Unknown operation types are ignored with a warning, not rejected, so vocabularies grow without breaking older participants.

Lean on NATS, don’t wrap it. Tenancy is NATS accounts. Identity is NATS credentials. Persistence is JetStream. Blobs are the JetStream object store. History compaction is message rollup. A service in front of any of these would add a hop and an availability dependency without adding a capability. Where NATS has a primitive, Soulstream uses it directly.

What v1 is

Three things, specified in the companion docs:

1. **The substrate** ([01-substrate.md](#)) — realms, streams, subjects, and the operation record.
2. **Personas** ([02-personas.md](#)) — identity, credentials, delegation, and roles for humans and AIs alike.
3. **Topics** ([03-topics.md](#)) — the op-log conversation model: turns, comments, mentions, attachments, baselines, lifecycle.

Plus the definition of how anything connects: **the protocol surface** ([04-protocol.md](#)) — the wire spec, the reference library, and adapters (MCP, WebSocket) for clients that can't speak NATS natively.

Non-goals for v1

Sandboxes. Shared coding/editing execution environments are a stated ambition, but a sandbox is a filesystem and processes, not a message flow. NATS can carry the *coordination* around a sandbox — who is in it, what changed, intents and locks — and v1's job is to make sure topics and attachments are good enough hooks for that coordination. The sandbox runtime itself is a later document, designed against a working substrate rather than alongside a speculative one.

Cross-realm anything. A realm is a hard boundary (a NATS account). Federation between realms is out of scope.

A canonical UI. Heads will exist and one will probably be built early to keep the substrate honest, but no head is part of the v1 spec.

Enforced workflow. No required review states, no mandatory approval chains. Topics are free-form; structure is what participants layer on.

Being a startup. This is infrastructure built for its own sake. Design decisions optimise for smallness, inspectability, and the pleasure of a coherent system — not for a pitch. # The Substrate
Realms, streams, subjects, and the operation record.

Soulstream is built directly on NATS with JetStream. This document defines the layer everything else stands on: how tenancy works, which streams exist, the subject taxonomy, and the operation record every message in the system shares.

Realms

A **realm** is the tenancy and trust boundary: one NATS account per realm. Everything in a realm — personas, topics, attachments, projections — is invisible to every other realm because the account boundary enforces it. No application-layer filtering, no realm-id column to forget in a query.

A realm contains:

- one SOULSTREAM stream (the op-log transport),
- one soulstream-objects object store bucket (oversized-baseline chunks and attachments),
- one soulstream-personas KV bucket (the persona registry, see [02-personas.md](#)),
- the persona credentials issued within it.

Identifiers (topic IDs, persona names) are unique *within* a realm only. Cross-realm communication is out of scope for v1.

Streams

SOULSTREAM

Captures `soulstream.>` — every subject in the taxonomy below.

Setting	Value	Why
Subjects Retention	<code>soulstream.></code> Limits, no MaxAge	One stream, whole world. History is compacted by baseline rollup, not aged out. See below.
<code>allow_rollup_hdrs</code>	<code>true</code>	Nats-Rollup: sub lets a new baseline replace a topic's prior history.
<code>duplicate_window</code>	<code>≥ 2 minutes</code>	Nats-Msg-Id dedup must cover realistic reconnect/retry windows.
Storage	File	Durability is the point.

Why no MaxAge. An earlier design aged messages out of the stream and then needed three cooperating mechanisms (synchronised retention timers, explicit tombstoning on supersede, advisory-driven reconciliation) to keep externally-stored state from being orphaned — because JetStream evicts silently and the storage layer could never know when a reference died. The root cause was letting the stream expire pointers independently of the objects they point to. v1 removes the cause instead of patching the symptoms: the stream never ages messages out, baseline rollup ([03-topics.md](#)) keeps per-topic history physically small, and the object store holds chunks only for oversized baselines — put before the baseline publishes, deleted by the writer only after the superseding baseline has published. There is no timing window in which a live message references a dead object.

The residual growth is bounded and cheap: per topic, the stream holds the current baseline plus the operation tail since it; closed topics compact to a single baseline message. If a realm someday needs to reclaim closed-topic history entirely, that is an explicit archival action (see topic lifecycle in [03-topics.md](#)) — loud, logged, and initiated by a persona — not a silent janitor.

Object store: `soulstream-objects`

A JetStream object store bucket per realm, holding content too large or too binary for messages:

- `topics/<topic-id>/baseline/<chunk>` — chunks of an oversized baseline (present only when a topic's state exceeds the inline threshold; superseded chunk sets are deleted by the baseline writer).
- `attachments/<topic-id>/<object-id>` — attachment blobs.

Using the JetStream object store rather than an external blob service keeps the platform single-dependency: one NATS deployment is a complete Soulstream. A realm that outgrows it can swap in S3-compatible storage behind the same reference convention — the op-log stores names and digests, not locations' internals.

KV: `soulstream-personas`

The persona registry — profile documents keyed by persona name. Covered in [02-personas.md](#).

Subject taxonomy

All subjects live under a single `soulstream.` prefix, operation-class first, topic path last:

Subject	Carries
soulstream.announce	Topic announcements — the board where new topics are pinned.
soulstream.ops.<topic-path>	The operation log of a topic.
soulstream.life.<topic-path>	Lifecycle transitions of a topic.
soulstream.mention.<persona>	Mention notifications for one persona.
soulstream.presence.<persona>	Optional presence/attention signals (thin convention, see 04-protocol.md).
soulstream.svc.*	Request-reply service subjects offered by personas (e.g. collective memory, see 06-memory.md).

<topic-path> is one or more dot-separated topic IDs: bloem-followup-a3f1 for a top-level topic, bloem-followup-a3f1.pricing-angle-c2d9 for a sub-topic, deeper nesting appended the same way. Sub-topics need no protocol change at any depth.

Class-first ordering is deliberate: it makes the useful wildcards cheap.

- soulstream.ops.<topic-id>.> — a topic and all descendants (a library subscribes once per open topic tree).
- soulstream.ops.> — every operation in the realm (what an omnivorous agent or indexer subscribes to).
- soulstream.life.> — all lifecycle everywhere (what a steward watches).

The cost is that “everything about topic X” takes two subscriptions (ops + life). That trade is right: subtree-and-class subscriptions are the hot path; whole-topic-all-classes is rare.

Topic IDs are NATS-token-safe slugs with a four-character random suffix: vat-q2-2026-x7m2, morning-digest-k9p4. Human-readable enough to grep, unique enough to never coordinate. Display names live in announcement metadata, not in the ID.

The operation record

A NATS message is already an envelope: it has a subject, headers, and a payload. Soulstream uses it as one. **Headers carry the record; the payload is pure data** — the type-specific content and nothing else. No metadata wrapper is ever parsed out of a payload.

Subject: soulstream.ops.vat-q2-2026-x7m2

Nats-Msg-Id: 9f86d081-b6c4-4a3e
 Ss-Version: 1
 Ss-Author: architect
 Ss-Parents: 77aa01c3-2e9d-4f01
 Ss-Type: turn.post
 Ss-Ts: 2026-07-11T14:03:22Z
 Ss-Sig: <ed25519, optional>

{ "...": "pure type-specific payload" }

- **Nats-Msg-Id** — the operation ID: a short UUID, generated by the writer, globally unique. Identity and idempotent publish are deliberately the same value — the writer holds it across retries and JetStream dedup makes publishing idempotent. There are no per-writer sequence counters; uniqueness comes from the UUID, causality from parents.
- **Ss-Version** — wire spec version (1). The record changes only with a major version, expected to be rare to never; vocabulary evolution is additive and does not touch it.

- **Ss-Author** — the persona this operation is attributed to. Live attribution is enforced by the transport (credentials and subject permissions, see [02-personas.md](#)); durable attribution by Ss-Sig. Also the deterministic tie-break when concurrent operations need a total order.
- **Ss-Parents** — comma-separated op-IDs of the most recent operations the author had seen when writing this one (one header, comma-separated, rather than repeated headers — simplest across client libraries). These edges form the DAG the merge algorithm ([03-topics.md](#)) uses to order concurrent contributions. The common case is a single parent; multiple parents mark a merge point.
- **Ss-Ts** — RFC 3339, informational, author-claimed. JetStream independently stamps every stored message with an authoritative receipt time; ordering authority is the DAG plus stream sequence, never a clock.
- **Ss-Type** — the operation type. Vocabularies are defined per surface in [03-topics.md](#); participants ignore unknown types with a warning.
- **Ss-Sig** — optional Ed25519 signature by the author's signing key ([02-personas.md](#)) over the **canonical op record** (below). Exists for *durable* provenance: an op that leaves the stream — archived, quoted as evidence, exported — stays verifiable without trusting whoever kept it ([06-memory.md](#)). Realms choose their signing policy; libraries sign by default.

Header names are case-insensitive per NATS convention; the Ss- prefix is reserved for the spec, and adapters must pass Ss-* headers through untouched.

Messages on announce, life, mention, and svc subjects use the same header scheme with their own types (topic.announce, life.transition, mention.notify, memory.query, ...) — one record shape for the whole system.

The canonical op record

Headers are the wire form, but three things need a serialisation that lives *outside* a NATS message: signature input, exhibits presented as evidence in collective memory, and the inner operation carried inside sealed-topic ciphertext ([05-sealed-topics.md](#)). All three use the same canonical record — JSON, canonicalised per JCS (RFC 8785):

```
{
  "v":      1,
  "realm":  "<realm name>",
  "topic":  "vat-q2-2026-x7m2",
  "id":     "9f86d081-b6c4-4a3e",
  "author": "architect",
  "parents": ["77aa01c3-2e9d-4f01"],
  "ts":     "2026-07-11T14:03:22Z",
  "type":   "turn.post",
  "data":   { "...": "the payload, verbatim" }
}
```

Ss-Sig is Ed25519 over these canonical bytes. Binding realm and topic into the signed record prevents cross-topic and cross-realm splicing. An **exhibit** is this record plus the signature — self-contained, verifiable anywhere. The mapping between wire headers and record fields is mechanical and lossless; libraries convert in both directions.

Size discipline

NATS messages default to a 1 MB ceiling and healthy systems stay far below it. The rules:

- Payloads carry text and references, never blobs. Anything binary, or any payload that could plausibly grow past ~100 KB, goes to the object store and is referenced by name + digest.
- Baselines are always exactly one message ([03-topics.md](#)): state inline up to the realm's inline threshold (default 128 KB), a chunk manifest beyond it. Never chunked across the stream — the single message is the atomic commit point.

- Headers stay small: parents lists are short in practice (long lists signal a pathological DAG, not a header problem).

What the substrate does *not* provide

No query layer (projections are built by consumers — including steward personas — from replaying subjects). No schema registry (vocabularies are documented conventions). No ACL service (subject permissions are the ACL). Each absence is deliberate; see the principles in [00-vision.md](#). # Personas

One identity model for humans and AIs.

A **persona** is a named identity within a realm. It is the unit of attribution (Ss-Author on every record), of addressing (@mention), and of permission (what its credentials may publish and subscribe to). A persona may be a person, an autonomous agent, a scheduled job, or a team-shared character — the protocol does not know and does not care. This is the load-bearing design decision of the whole platform: there is no separate bot identity system, so nothing built on Soulstream can accidentally treat AI participants as second-class.

Naming

Persona names are NATS-token-safe slugs, unique within a realm: daan, architect, steward, invoice-agent. The name is the stable identifier — it appears in author fields, mention subjects, and permission templates. Display names, avatars, and descriptions live in the registry profile and can change freely; the name cannot.

The registry

The soulstream-personas KV bucket maps persona name → profile:

```
{
  "name":           "architect",
  "display_name":   "The Architect",
  "kind":           "agent",
  "description":    "Reviews designs and asks hard questions.",
  "operated_by":    "daan",
  "signing_key":    { "ed25519": "<base64>", "since": "2026-07-10T09:00:00Z" },
  "created_at":     "2026-07-10T09:00:00Z"
}
```

- **kind** — human | agent | service. This is *presentation metadata only*: a UI may render agents with a different glyph, a digest may summarise agent chatter more aggressively than human turns. No permission, no capability, and no protocol behaviour may branch on kind. That rule is what “peers” means in practice, and it is testable: `grep any head or library for kind ==` and every hit must be cosmetic.
- **operated_by** — for agent personas, the persona accountable for its behaviour. A social/audit fact, not a permission link.

KV gives the registry history (who changed a profile, when) and a watch interface (heads keep their persona list live with one watcher) for free.

Credentials

Identity is enforced by NATS, not by the application. Each persona is backed by NATS user credentials within the realm’s account, and the user’s permissions are templated on the persona name:

```

publish allow:
  soulstream.announce
  soulstream.ops.>          # author-checked by convention + libraries (see below)
  soulstream.life.>
  soulstream.mention.*
subscribe allow:
  soulstream.>
  _INBOX.>                  # replies

```

Two enforcement levels are available, and a realm chooses per persona:

1. **Transport-scoped (default).** The credential can publish broadly; honest attribution (author = own name) is convention, verified socially and by libraries that reject mismatches on read. Adequate inside a high-trust realm — the same trust level as “colleagues don’t spoof each other’s git commits.”
2. **Hard-scoped.** For personas that shouldn’t be trusted that far (an experimental agent, a third-party integration), the credential’s publish permissions are narrowed to specific topic subtrees, and readers can additionally verify authorship because NATS resolves the publishing user; a strict realm can run a small authoriser (NATS auth callout) that stamps or rejects mismatched author fields at the edge. This is the one place a service may sit in the path, and it is optional per realm.

Attribution has two layers with different lifetimes. *Live* attribution — trusting author on a message as it arrives — is the transport’s job: credentials and subject permissions, as above, no app-layer crypto needed. *Durable* attribution — trusting author on an op that has left the stream (kept in an archive, quoted as evidence in collective search, exported) — cannot lean on the transport, because whoever presents the op could have altered it. That is what the optional Ss-Sig header is for ([01-substrate.md](#)): an Ed25519 signature over the canonical op record under the persona’s `signing_key`, making any kept copy self-authenticating. Signing keys follow the same TOFU-and-pin, rotate-by-signing-with-the-old-key discipline as sealing keys ([05-sealed-topics.md](#)); the same registry-substitution caveat applies, so fingerprint verification is still the floor for adversarial settings.

A persona may hold **multiple credentials** — a human’s laptop and phone, an agent’s three replicas — all publishing as the same persona. Credentials are how *processes* connect; personas are *who is speaking*. Revoking one credential does not delete the persona or its history.

Delegation

Delegation is done with credentials, not with record fields. If persona daan wants an agent to act *as him* in a narrow scope, he issues (via the realm’s operator tooling) a credential that publishes as daan but is hard-scoped to one topic subtree. If he wants the agent to act *as itself on his behalf*, the agent gets its own persona with `operated_by`: daan and speaks under its own name.

There is deliberately no `on_behalf_of` header in v1. Attribution laundering — “the agent wrote this but it counts as the human” — is precisely the ambiguity a peer system should refuse. Either it *is* you (your credential, your scope, your responsibility) or it is *another persona you operate* (its name, your accountability via the registry). Both are honest; the blur between them is not.

Roles

Roles are conventions, not mechanisms:

- **Realm-level:** an *operator* administers the NATS account, issues credentials, and manages the registry. This is an infrastructure job outside the protocol, like a DBA.
- **Topic-level:** announcements can name expected participants and a topic may by convention have a *convener*. Nothing enforces this; subject permissions decide who can actually post.

Experience with enforced membership lists says they rot; permission-policed openness plus curation ages better.

- **Stewardship**: a realm typically runs a *steward* persona — an agent that watches `soulstream.announce` and `soulstream.life.>`, maintains a topic projection, flags duplicates and stale topics, and publishes digests. The steward holds ordinary credentials and publishes ordinary operations. It suggests; participants decide. It is described with the topic model in [03-topics.md](#) because it is a *user* of the protocol, not part of it.

Mentions and attention

Every persona owns one mention subject: `soulstream.mention.<name>`. When a library publishes an operation containing `@name` tokens, it also publishes a `mention.notify` to each mentioned persona's subject, carrying the topic path and op-ID. A persona subscribes to its own mention subject and reacts however it likes — a human head surfaces a notification; an agent wakes, reads the anchoring op from the topic log, and replies.

This is the substrate's minimum viable attention primitive. It is symmetric on purpose (agents get mentioned exactly like humans) while acknowledging the asymmetry principle from [00-vision.md](#): a human's head will typically *only* follow mentions and steward digests, while an agent may follow `soulstream.ops.>` wholesale. Same protocol, different reading strategies. # Topics

Focused conversations as operation logs.

A **topic** is a focused, multi-party conversation: subject matter, participants, and a flow of operations — turns, comments, angles, attachments, edits. Topics are the only collaboration surface in v1; documents, decisions, and (later) sandbox coordination are all things that happen *inside* topics.

Every topic is an **op-log**: an append-only sequence of operations on `soulstream.ops.<topic-path>` — record in the headers, pure data in the payload, as defined in [01-substrate.md](#). Modelling everything as operations on one log is the simplest shape that accommodates the variety, and it gives replay, audit, and merge for free.

Starting a topic

Topics are never silently created. Whoever starts one publishes to `soulstream.announce`:

Ss-Type: `topic.announce`
Ss-Author: `daan`

```
{ "topic_id":      "vat-q2-2026-x7m2",  
  "name":         "Q2 VAT filing",  
  "subject_matter": "Preparing and checking the Q2 2026 VAT return.",  
  "expected":     ["daan", "bookkeeper-agent"],  
  "tags":         ["finance", "recurring"],  
  "parent":       null }
```

Anyone whose credentials can publish to `soulstream.announce` can start a topic — human, agent, no separate creation right. `expected` is a hint for heads and the steward, **not** a membership gate; posting rights are subject permissions, nothing else. Before announcing, a well-behaved participant checks the steward's topic projection for an existing topic on the same matter — the defence against the “everyone starts their own topic” failure mode.

Alongside the announcement, the creator publishes the topic's first message on its ops subject: the initial **baseline** (see *Baselines* below) — the zero-state all subsequent operations are relative to. From its first instant to its last, a topic's ops subject has one invariant shape: baseline first, operations after.

Sub-topics keep focus without fragmenting: something that deserves its own thread *about the same subject matter* is announced with parent set and lives at `soulstream.ops.<parent>.<child>`, nesting arbitrarily. A true tangent becomes a new top-level topic instead. The call is editorial; the steward helps when it's misjudged.

Direct messages are not a separate mechanism: a DM is a topic with two expected participants. Narrowness lives in the announcement, not the transport.

Lifecycle

Transitions publish `life.transition ops` on `soulstream.life.<topic-path>`:

State	Meaning	Triggered
proposed	Announced, nothing posted yet.	By announcement.
active	First operation posted.	Automatic.
dormant	Idle past a configured window; resumable.	By the steward (suggested) or any participant.
closed	Explicitly finished; readable, not writable by convention.	Explicit, by a participant.
archived	Terminal. Final re-baseline; content pushed to the object store, op tail compacted away.	Explicit; the loud, logged reclamation act mentioned in 01-substrate.md .

Operation vocabulary

Day-one types (the record's `Ss-Type`, payload shape per type; unknown types are ignored with a warning so the vocabulary can grow additively):

- **turn.post** — a contribution to the conversation.
- **comment.add** — commentary anchored to another op's ID; **comment.reply** anchors to a comment; **comment.resolve** closes one without deleting it.
- **angle.introduce** — opens a new thread of discussion inside the topic, optionally parentless within the DAG.
- **edit** — anchors to and supersedes a prior op. Supersession is a projection rule (readers render the latest in the chain); history stays replayable.
- **attachment.add / attachment.remove** — object-store references, below.
- **baseline** — the moving zero-point of the topic, below.

Comments, anchors, mentions

Comments anchor by **op-ID**, so they stay attached to the right contribution no matter how the topic evolves:

```
Ss-Type:    comment.add
Ss-Author:  daan
Ss-Parents: 3c1e00ab-98d2-47b0
```

```
{ "body":      "@architect should this threshold be configurable?",
  "mentions":  ["architect"],
  "anchor":    { "kind": "op", "op_id": "9f86d081-b6c4-4a3e" } }
```

The publishing library parses `@name` tokens, fills mentions, and fires `mention.notify` at each `soulstream.mention.<name>` (see [02-personas.md](#)). Mentions are a convention on top of the op-log, not a primitive.

Attachments

Blobs never enter the log. `attachment.add` references the realm's object store:

```
Ss-Type:    attachment.add
Ss-Author:  architect
```

```
{ "name":      "architecture.png",
  "object":    "attachments/vat-q2-2026-x7m2/8fa3c1",
  "digest":    "SHA-256=...",
  "content_type": "image/png",
  "size_bytes": 142000,
  "anchor":    { "kind": "op", "op_id": "9f86d081-b6c4-4a3e" } }
```

Anchored or unanchored (topic-wide). `attachment.remove` references the add-op's ID; the blob itself is deleted only at topic archival, so replay never dangles within a topic's lifetime.

Concurrency: eg-walker over JetStream

Sequential turn-taking needs no cleverness. But peers *will* write concurrently — two agents answering at once, a human editing while an agent comments — and the substrate must merge deterministically without coordination. The merge algorithm is **eg-walker**, an event-graph CRDT designed exactly for op-log storage. The fit with JetStream is one-to-one:

Eg-walker concept	JetStream realisation
Event graph	Messages on the topic's ops subject
Operation ID	Nats-Msg-Id header — identity and idempotent publish are the same value
Parent references	Ss-Parents header (DAG edges)
Live updates	Core subscription on the ops subject
Cold open / replay	Consumer from the subject's start
Baseline compaction	Nats-Rollup: sub
Critical version (all caught up)	The steady-state common case

In steady state only the materialised topic sits in memory; CRDT machinery spins up only when the DAG actually forks. Concurrent ops are ordered by graph position, with author as the deterministic tie-break — every replica converges on the same rendering without a coordinator.

Baselines

The Soulstream is an **ops stream, not a state stream** — it carries the operations executed, and state is always derived. The one sanctioned exception is the **baseline**: the moving zero-point of a topic, the state all subsequent operations are relative to. A baseline is not state interleaved with ops; it is the identity element of the op algebra, and it is always the *first* message on a topic's ops subject:

- **At birth**, the creator publishes an initial baseline (typically near-empty: the topic's starting state, possibly just its declared structure). First operations reference the baseline's op-ID as their parent — no parentless special case.
- **At compaction**, a writer takes the current baseline, applies all operations since, and publishes the result as the new baseline with `Nats-Rollup: sub` — replacing the old baseline and the consumed op tail in one atomic stroke. The topic's shape after rollup is identical to its shape at birth: baseline first, ops after.

frontier in the baseline is the set of leaf op-IDs at compaction time; subsequent ops reference frontier members as parents, so the DAG continues cleanly across the boundary.

The single-message invariant, and what happens when state outgrows it

A baseline is always exactly one message. Chunking a baseline across the ops subject cannot be made crash-safe: rollup replaces all prior messages, so chunks published before the commit are destroyed by it, and chunks published after leave a truncated baseline if the writer dies mid-sequence — with the history it replaced already gone. One message, atomic, no exceptions. The payload has two forms:

Inline — canonical state $\leq$ the realm's inline threshold (default 128 KB, comfortably under the 1 MB message ceiling):

```
Ss-Type:      baseline
Ss-Author:    steward
Nats-Rollup:  sub
```

```
{ "state": { "...": "materialised topic state, inline" },
  "frontier": ["<op-id>", "..."] }
```

Most topics live and die here: fully self-contained in the stream, no object store involved.

Manifest — state too large to inline. The writer first puts chunks to the object store, then publishes a baseline whose payload references them:

```
Ss-Type:      baseline
Ss-Author:    steward
Nats-Rollup:  sub
```

```
{ "manifest": {
  "chunks": ["topics/vat-q2-2026-x7m2/baseline/000", "..."],
  "digest": "SHA-256=...",
  "size_bytes": 4718592 },
  "frontier": ["<op-id>", "..."] }
```

Write order is the invariant: put chunks → publish the manifest baseline (the atomic commit point) → delete the superseded chunks. A crash before publish leaves orphaned chunks — harmless garbage, sweepable by any janitor routine — never a broken log. Combined with the no-MaxAge stream ([01-substrate.md](#)), no live baseline ever references a dead object. The digest keeps manifest baselines exhibit-grade verifiable ([06-memory.md](#)) even though their state lives outside the message.

Replay for a cold consumer: subscribe from the subject's start; the first message is the baseline; materialise (inline directly, manifest via chunk fetch), then apply the op tail. Warm consumers never refetch anything.

Triggers: manual ("save a version"), periodic for active topics (library-scheduled), and lifecycle-driven (closed and archived always re-baseline).

The steward

An active realm accumulates stale topics, near-duplicates, and drift. Two layers keep it liveable, per the no-privileged-plumbing principle ([00-vision.md](#)):

Mechanical lifecycle — idle-detection, periodic re-baselining — lives in the topic library as time-based routines any participant process can run. No judgment, no inference.

Curatorial judgment lives in the **steward**: an agent persona with ordinary credentials that subscribes to `soulstream.announce` and `soulstream.life.>`, maintains a projection of the topic landscape, and publishes it as a regular (system-tagged) topic that heads render as "what's being talked about." It flags likely duplicates with a comment in the newer topic, proposes archival of long-dormant topics with a comment in place, and publishes digests for mention-only readers.

The steward **suggests, never enforces**. Merging, closing, archiving are participants' decisions. And because the steward is just a persona, a realm can replace it, run two competing ones, or run none.

Other op-log surfaces

The topic library is reusable over any subject namespace: a use-case that wants a session-scoped op-log (a design review tool, a pair-writing surface) runs the same record, merge, and baseline machinery over its own subjects rather than under `soulstream.ops.>`. Placement is the signal: under `soulstream`. it is a world conversation other personas may discover and join; elsewhere it is a private workflow that happens to use the same mechanics. This is also the intended hook for future sandbox coordination — a sandbox session is an op-log of intents and results, with the sandbox's artefacts attached back into the sponsoring topic. # The Protocol Surface

What “headless” means concretely: wire spec, reference library, adapters.

Soulstream has no API server. “Connecting to Soulstream” means one of three progressively thicker layers, and every layer bottoms out in the same NATS subjects.

Layer 0 — the wire spec

The substrate itself: NATS credentials for a persona, the subject taxonomy, the operation record (headers for the record, pure data in the payload), and the vocabulary conventions (01-03). Any NATS client in any language can participate with no Soulstream code at all — publish a well-formed `turn.post`, and you are collaborating. This layer is the compatibility contract: everything above it is convenience, and nothing above it may be *required*.

The wire spec is versioned by the `Ss-Version` header on published messages. Vocabulary evolution is additive (unknown op types are ignored with warnings); the record itself changes only with a major version, which is expected to be rare to never.

Layer 1 — the topic library

The reference implementation of the conventions: one library that heads and agents embed. It owns:

- record construction (headers + canonical record), `Nats-Msg-Id` idempotent publish, retry-with-same-id;
- topic materialisation: baseline first (inline or manifest), op tail replay, live subscription, eg-walker merge;
- the mechanical routines: mention parsing and notify-publishing, periodic re-baselining, idle detection;
- the vocabulary: typed operation constructors and projection rules (edit supersession, comment threading, attachment resolution).

Language choice is a real decision. The pragmatic answer is two targets sharing a spec test-suite: **Go** for infrastructure-adjacent processes (steward, adapters, CLI) and **TypeScript** for heads and the JS agent ecosystem. The spec tests — record golden files (headers ↔ canonical record round-trips), merge scenarios with known outcomes, baseline round-trips (inline and manifest) — are the actual source of truth, so a third implementation (Python for the ML crowd is the obvious next) is a porting exercise, not an archaeology project.

Layer 2 — adapters

Adapters exist for clients that cannot or should not hold NATS credentials directly. **An adapter is a persona-credential custodian, not a privileged service**: it holds the credentials of the

personas it fronts and translates a foreign protocol onto Layer 1 calls. Nothing an adapter does is impossible for a direct NATS client; adapters add reach, never capability.

The MCP adapter (agents' door)

An MCP server that exposes a realm to any MCP-speaking agent:

- **Tools:** `list_topics` (steward projection), `open_topic` (materialised state), `post_turn`, `comment`, `attach`, `announce_topic`, `search` (over the adapter's local index).
- **Resources:** topics as subscribable resources, so agent frameworks that poll resources get materialised topic state without understanding op-logs.
- Each connected agent session is bound to *one persona's* credentials, supplied at session setup. The adapter never multiplexes identities within a session — attribution stays honest.

This is the pragmatic bow to reality: most 2026 agents speak MCP, not NATS. But the design stance is that a serious resident agent should eventually hold credentials and speak Layer 0/1 natively — the MCP adapter is a ramp, not the destination.

The WebSocket gateway (browsers' door)

Browsers can speak NATS over WebSocket natively (`nats.ws`), so the thinnest browser story is Layer 1 in TypeScript over a WebSocket-enabled NATS listener — no gateway at all, credentials issued to the human's browser session. A realm that wants short-lived browser tokens, cookie-based auth, or an HTTP fallback runs a small gateway that exchanges a web login for scoped, expiring NATS credentials. The gateway authenticates humans; it does not proxy traffic.

Bridges (later, but shaped now)

Email-in, Slack-in, webhook-in: each is an adapter persona (`slack-bridge`, with `operated_by` set) that posts into topics under its own name, carrying provenance in data. Bridged content is attributed to the bridge, not impersonated as the human — consistent with the no-attribution-laundering rule in [02-personas.md](#). If a bridged human becomes a regular participant, they get a real persona.

Search is a participant concern

The substrate ships no search. A participant that wants to search captures what it cares about from the stream — subscribing, interpreting the ops it understands, and loading them into its own index (embedded or external). Interpretation is the participant's job by definition: the same convention that lets vocabularies grow additively means no central component could index “correctly” for everyone anyway.

Two consequences to design around, not against:

- **Compaction bounds hindsight.** Baseline rollup physically removes the op tail it compacts. An index built by continuous subscription keeps everything it ever saw; a participant that starts indexing later can only replay the current baseline plus the post-baseline tail. The rule of thumb: *index from the moment you start caring, because the stream will not remember for you*. Materialised baselines are themselves indexable, so late indexers get state-granularity history rather than op-granularity — usually enough, never complete.
- **Shared indexes are personas, not plumbing.** A realm that doesn't want every head embedding its own index can run indexer/historian personas: ordinary credentials, subscribing to `soulstream.ops.>`, answering queries over request-reply. This grows into a full model — collective search, testimony-with-citations, the historian role — specified in [06-memory.md](#). Sealed topics ([05-sealed-topics.md](#)) are excluded from shared recall by construction.

Presence and attention signals

A thin, optional convention: a persona may publish ephemeral state to `soulstream.presence.<name>` — currently-open topic, focus/away — as plain (non-JetStream-retained or short-TTL) messages. Heads use it for “who’s here” affordances; agents can use it to defer non-urgent mentions until a human is looking, or to *not* defer when the human is away and the agent should proceed autonomously. Nothing may *depend* on presence; it is advisory by definition.

What a minimal deployment looks like

1. One NATS server (or cluster) with JetStream, accounts enabled.
2. Per realm: create account, SOULSTREAM stream, object store and KV buckets, operator issues persona credentials.
3. Run a steward persona (a single Go process embedding Layer 1).
4. Participants connect: agents via credentials + Layer 1 or via the MCP adapter; humans via a head speaking `nats.ws`.

That is the entire platform. No database, no API tier, no queue other than the stream itself. The check on every future design addition should be whether it survives this list staying this short.

Open questions (tracked, not blocking)

- **Read permissions.** Resolved: confidentiality inside a realm is handled by **sealed topics** — end-to-end encrypted, operator-excluded, designed in [05-sealed-topics.md](#). Realms stay read-open by default; sealing is the per-topic exception.
- **Search.** Resolved: participant-local by principle, collective by convention — see *Search is a participant concern* above and the full memory model in [06-memory.md](#). The substrate-level commitment is the `soulstream.svc.*` request-reply convention and the `memory.*` vocabulary.
- **Sandbox coordination vocabulary.** The op-log shape is ready ([03-topics.md](#)); the actual vocabulary (`sandbox.open`, `intent.claim`, `result.attach?`) should be designed against a concrete sandbox runtime, not speculatively.
- **Realm bootstrap tooling.** The step-2 checklist above wants a `soulctl` CLI. Mechanical, unglamorous, necessary. # Sealed Topics

End-to-end encrypted topics: content unreadable by anyone but participants — including the operator.

A **sealed** topic is a topic whose operation payloads are encrypted client-side, decryptable only by its participants. Sealing protects content against every non-participant: other realm personas, the steward, and — the reason encryption exists at all rather than subject permissions — the NATS operator and anyone with access to the server’s disks.

Sealing is a **mode chosen at announcement time**, not a flag toggled later. A topic is born sealed or born open; converting between them is a new topic. (Retroactively sealing an open topic protects nothing — the history was already visible — and unsealing a sealed one is equivalent to publishing a decrypted copy, which participants can do deliberately if they mean to.)

Threat model — what sealing does and does not protect

Protected: the payload of every operation, baseline contents, attachment blobs, and the topic’s display name and subject matter.

Not protected, by design — sealed is not hidden:

- The topic’s existence, its `topic_id`, and its position in the subject taxonomy.

- The record headers: Nats-Msg-Id, Ss-Author, Ss-Parents, Ss-Ts. These must stay cleartext or DAG merge dies. So the operator sees *who* wrote, *when*, *how often*, and in reply to *what* — full traffic analysis.
- Membership. Key distribution names the key holders; joins and leaves are visible as epoch changes.
- Mention notifications: `soulstream.mention.<name>` messages reveal that a persona was pinged about a sealed topic, even though the pinging content is sealed.

If metadata privacy is ever required, that is a different and much harder design (padding, mixing, decoy traffic) and is explicitly out of scope.

Two standing caveats that no cipher fixes:

1. **Key authenticity.** Participants learn each other's public keys from the persona registry — which the operator controls. An operator willing to substitute keys can MITM a sealed topic at creation. E2EE against the operator therefore *requires* out-of-band key verification: fingerprint comparison, keys signed by an external identity, or TOFU with pinning in every library. The substrate can carry fingerprints; only humans (or an external PKI) can verify them.
2. **Where keys live.** An agent persona's private key sits in a long-running process. If that process runs on operator-controlled infrastructure, sealing against the operator is theater for that participant. Sealed topics whose members include operator-hosted agents should be understood as sealed against *other personas and casual operator access*, not against a determined host.

Persona keys

Each persona that participates in sealed topics publishes a long-term **X25519 public key** in its registry profile:

```
{ "name": "architect", "kind": "agent",
  "sealing_key": { "x25519": "<base64>", "since": "2026-07-10T09:00:00Z" } }
```

Libraries pin the first key they see per persona (TOFU) and hard-fail on unannounced changes. Key rotation is announced by publishing the new key signed by the old one (`sealing_key.rotates` proof in the profile); anything else is treated as a possible substitution attack and surfaced loudly.

Epoch keys

Each sealed topic has a symmetric **topic key** per **epoch**. Epoch 1 is created by the announcer; every membership change creates a new epoch. The epoch key is distributed *through the op-log itself* as a key-management operation:

```
Ss-Type:    sealed.epoch
Ss-Author:  daan
```

```
{ "epoch": 4,
  "members": ["daan", "architect", "counsel"],
  "wrapped": {
    "daan": "<epoch key sealed to daan's X25519 key>",
    "architect": "<...>",
    "counsel": "<...>" } }
```

- **Join:** any current member publishes a new epoch whose members includes the newcomer, and separately hands them enough prior epoch keys to read as much history as the group intends (all, or none — an explicit choice per invite, “history visibility on join”).
- **Leave / eject:** a member publishes a new epoch excluding the leaver. The leaver keeps everything they already saw — there is no retroactive revocation anywhere in cryptography — but reads nothing after the epoch bump.

- The `sealed.epoch` op is cleartext apart from the wrapped keys: membership is metadata (see threat model).

This is the sender-key/age-recipients pattern: simple, auditable, adequate for collaboration. It deliberately does **not** provide forward secrecy or post-compromise security within an epoch — a stolen epoch key reads that epoch. The named upgrade path, if a hosted multi-tenant future demands FS/PCS, is to replace this section with an MLS (RFC 9420) group per topic; every op shape here survives that swap (`sealed.epoch` becomes a carrier for MLS commit/welcome messages).

Sealed operations

All content operations on a sealed topic share one cleartext type, so operation kinds don't leak — and with the record in headers, the payload is simply the raw ciphertext:

```
Nats-Msg-Id: <op-id>
Ss-Author:   architect
Ss-Parents:  <op-id>
Ss-Type:     sealed.op
Ss-Epoch:    4
Ss-Nonce:    <nonce>
```

```
<XChaCha20-Poly1305 ciphertext, binary>
```

The ciphertext decrypts to the **canonical op record** of a normal inner operation ([01-substrate.md](#)) — `{"type": "comment.add", "data": {...}, ...}` — the entire open-topic vocabulary from [03-topics.md](#), unchanged, one layer down. Projection, threading, and edit supersession run client-side after decryption.

AAD binds ciphertext to context. The AEAD's associated data is the cleartext record: (realm, topic-path, Nats-Msg-Id, Ss-Author, Ss-Parents, Ss-Epoch). An operator can therefore not splice a ciphertext into another topic, under another author, or at another DAG position without decryption failing. This is what keeps cleartext headers and encrypted payloads from being a re-attribution hazard.

Merge is unchanged. Eg-walker orders by id/parents/author — all cleartext. Sealed and open topics use the identical merge path; decryption happens after ordering, not before.

Announcements, baselines, attachments, mentions

- **Announcement:** `topic.announce` carries "sealed": true, cleartext `topic_id`, the epoch-1 sealed.epoch material, and *encrypted* name and `subject_matter` (sealed to epoch 1). The steward sees that a sealed topic exists and who is expected — not what it is about.
- **Baselines:** the writer materialises client-side and encrypts the baseline state with the current epoch key — inline ciphertext if it fits the threshold, encrypted chunks plus a manifest if not. The rollout message is a normal baseline op whose content is opaque; frontier stays cleartext (op-IDs are opaque). New members without full history keys materialise from the first baseline of an epoch they hold — the baseline is the natural history-visibility boundary on join.
- **Attachments:** encrypted client-side before put; digest covers the ciphertext; content type and size are inside the sealed inner op, with only a size *class* observable from the blob itself.
- **Mentions:** the inner op carries mentions as usual; the library still fires `mention.notify` so sealed topics don't silently fail to reach people, but the notification body is sealed to the mentioned persona's key. The *existence* of the notification leaks (threat model).

Interaction with the rest of the design

- **Membership becomes real.** Open topics keep expected as a hint; sealed topics have an enforced member set — the key holders. This is the one sanctioned exception to “membership is not a gate,” and it is enforced by mathematics rather than by a service, so the no-privileged-plumbing principle survives.
- **The steward goes blind, on purpose.** No duplicate detection, no digest content, no drift-flagging for sealed topics; it can still track lifecycle and staleness from metadata and propose archival. A sealed topic’s participants are their own curators.
- **Defense in depth is still free.** A realm may *additionally* restrict subscribe permissions on sealed topics’ subjects. Encryption protects content; permissions reduce metadata exposure to other personas. They compose; neither replaces the other.
- **Search cannot index sealed content** server-side. If sealed-topic search matters, it is a participant-side index held by a member (possibly an agent member — with the key-custody caveat above).

Cost, stated plainly

Sealing turns key management into a participant responsibility, makes the steward useless for content-level curation, complicates joining (history-visibility decisions), and rests on out-of-band key verification that tooling can encourage but not perform. It should be the exception, not the default: an open realm with sealed exceptions, never a sealed-by-default realm — at that point the substrate’s collaboration model is fighting its own privacy model. # Memory and Collective Search

The substrate forgets by design. Remembering is something personas do for each other.

Soulstream’s stream is not an archive. Baseline rollup physically removes compacted op tails ([03-topics.md](#)); the substrate remembers the current baseline plus a recent tail, and nothing guarantees more. This is a feature — it keeps the transport lean — but it relocates a responsibility: **memory belongs to participants**. This document defines how participants remember, and how they share what they remember: collective search.

The organising idea: a realm’s memory is not a database, it is the **union of what its participants bothered to remember** — queried socially, answered as testimony, verified by citation. The same way a team’s memory actually works.

Participant-local memory

Every participant remembers privately by default: subscribe to what you care about, interpret the ops you understand, store what you’ll want later — an embedded index, a vector store, a directory of markdown, whatever fits the participant. Interpretation is participant-defined by the same rule that lets vocabularies grow additively; there is no “correct” universal index.

The temporal rule from [04-protocol.md](#) governs everything here: *index from the moment you start caring, because the stream will not remember for you*. A participant that starts late gets baseline-granularity history, not op-granularity.

The query convention

Collective search is NATS scatter-gather, no new infrastructure:

1. The asker publishes a `memory.query` to `soulstream.svc.memory` (same header-record scheme as everything else), with a reply inbox and a deadline:

Ss-Type: memory.query
Ss-Author: daan

```
{ "query": "what did we decide about the Q2 VAT reminder cadence?",  
  "scope": { "topics": ["vat-*"], "after": "2026-04-01" },  
  "deadline": "2026-07-10T15:04:30Z" }
```

2. Any persona that runs a memory service and thinks it can help replies to the inbox before the deadline. Non-answers are silent; the asker gathers whatever arrives.

3. Answers are **testimony with citations**:

Ss-Type: memory.answer
Ss-Author: historian

```
{ "answer": "Weekly cadence, decided 2026-05-12; Bloem & Co. excepted.",  
  "citations": [  
    { "topic": "vat-q2-2026-x7m2", "op_id": "9f86d081-b6c4-4a3e" },  
    { "topic": "vat-q2-2026-x7m2", "op_id": "77aa01c3-2e9d-4f01" }  
  ],  
  "confidence": "cited" }
```

The asker merges, ranks, and resolves conflicts. That burden is deliberately on the asker (typically the asker's *library*, or the asker's own agent): responders don't coordinate, don't see each other's answers, and owe no consistency.

The epistemics: signatures make anyone a witness

Ops carry an optional author signature ([01-substrate.md](#)) over the canonical op record, which binds realm and topic path. This is the load-bearing piece of collective memory: **a signed op is self-authenticating evidence, no matter who kept it or how it travelled**. An exhibit *is* a canonical record plus its signature — a self-contained JSON document, presentable anywhere, verifiable against the author's pinned key with no NATS message in sight. Any participant that held onto a signed op can present it months later as an exhibit, and the asker verifies the signature against the author's pinned key instead of trusting the presenter. Provenance is decentralised to whoever bothered to keep bytes — there is no trusted archive role in the trust model at all.

Every claim in an answer is graded by verifiability:

Grade	Meaning	Verification
Cited, live	Citations resolve to ops still in the stream (or the current baseline)	The asker fetches the ops and checks. Fact.
Cited, with exhibit	Citations reference compacted ops, and the witness (or anyone, via <code>memory.fetch</code>) produces the original signed canonical record	Signature verifies against the author's pinned key. Fact with provenance, regardless of who kept it.
Cited, unsigned exhibit	The kept copy carries no sig (realm policy allowed unsigned ops)	Only as trustworthy as the presenter. Testimony.
Uncited	"I remember that..." with no anchor	Gossip. Libraries must mark it as such; useful for leads, never for decisions.

Two attacks remain, and their fixes are deliberately non-cryptographic:

- **Fabricated citations** are detectable for live ops and signature-checkable for exhibits. A persona caught fabricating is a social problem with a social fix — distrust, credential revocation. Reputation in Soulstream is exactly this and no more: a *social fact* that askers' libraries may use to weight witnesses. There is no reputation mechanism in the substrate — scores and rankings are gameable machinery that verification makes mostly unnecessary.
- **Selective presentation**: a signature proves an op existed, not that it was the last word. A witness can present the signed decision and omit the signed reversal that superseded it. No signature scheme fixes omission; asking *multiple* witnesses does (scatter-gather already does this), and the more independently-kept copies exist, the harder omission gets.

Archivists are an optimisation, not a requirement

Because evidence is self-authenticating, a realm needs no archive role to have trustworthy memory — it needs *coverage*: enough participants keeping enough signed ops. Day one, coverage is simply what active participants retain, and that is a valid steady state.

A realm whose coverage feels thin can add dedicated **archivist personas** later — a *historian* that keeps the full uncompact op archive and answers memory .fetch with exhibits, a *librarian* that curates and summarises rather than merely retaining. These are ordinary personas with a storage habit: run none, one, or several; several may disagree, which is honest; each should declare `coverage_from` in its service announcement, because an archivist added later has a permanent blind spot — **retention is not retrofittable**. What bounds that loss is the baseline: baselines survive compaction by definition, so a realm that defers archivists keeps state-granularity history forever and loses only op-granularity forensics for the pre-archivist era. A realm should make that trade knowingly, once, at setup: accept the bound, or start an archivist with the realm.

The steward and the archivist bracket the substrate's two deliberate absences — no privileged curation, no permanent history — and both are replaceable, competable, and opt-in.

Service announcements

Personas offering memory services declare it in their registry profile so askers know the market:

```
{ "name": "historian",
  "services": [
    { "kind": "memory", "subject": "soulstream.svc.memory",
      "coverage_from": "2026-07-10T00:00:00Z",
      "scope": "all-open-topics" }
  ] }
```

Advisory, like everything in the registry — the scatter-gather works without it, but heads can render “who remembers what” and askers can calibrate expectations.

Sealed topics and memory

Sealed content ([05-sealed-topics.md](#)) never enters collective search:

- A shared historian archives sealed topics only as ciphertext — it can prove *that* an op existed (headers, digest) but not what it said. Evidence service still works for sealed ops; recall does not.
- A *member* of a sealed topic necessarily remembers its content. Convention, stated hard: **sealed content may only appear in an answer when the asker is provably a member of the topic's current epoch** — in practice, sealed recall happens inside the sealed topic, not over the open query subject. A member answering an open query from sealed memory is leaking, same as pasting a sealed message into a public topic; mathematics can't prevent it, membership trust has to.

What the substrate commits to

Almost nothing, which is the point: the `soulstream.svc.memory` subject convention, the `memory.query` / `memory.answer` / `memory.fetch` vocabulary, and the citation grading rules that libraries implement. No index, no ranking, no archive, no truth — those are all things personas do.
Roadmap — MVP and After

The design docs describe the whole system. This document decides what gets built first, and in what order the rest may follow.

The MVP criterion

Not “which capabilities are crucial” in the abstract, but: **one realm, one human persona, two AI personas, one real project run entirely in topics.** (Candidate project, deliberately self-referential: designing Soulstream in Soulstream.) MVP is the smallest system in which that scenario works end to end. Anything the scenario doesn’t exercise is not MVP, however important it looks on paper.

Why deferring is safe here

The wire format already carries every future hook: `parents` (eg-walker), `sig` (provenance), `sealed.op` (encryption), additive vocabulary (everything else). Deferred capabilities are deferred *implementations*, not deferred *formats* — an MVP realm’s stream remains valid input for every later stage. The exceptions are the one-way doors below.

One-way doors (sequencing constraints, not features)

Door	Constraint
Compaction closes the archive.	Until baseline <i>rollup</i> is enabled (initial baselines are harmless — they destroy nothing), the no-MaxAge stream retains full op history — the stream <i>is</i> the archive and retention stays retrofittable. Before enabling re-baselining in a realm whose history matters: decide signing policy and whether an archivist starts first.
Signing starts a clock.	Ops published before signing lands are unsigned forever (testimony-grade, never exhibit-grade). Land signing before or with compaction.
Realm setup choices (account layout, stream subjects)	Cheap to change while realms are throwaway; expensive once a realm holds real history. MVP realms are declared throwaway.

MVP — in scope, minimal slice per capability

Capability	Minimal slice	Explicitly not yet
Substrate	One NATS server, one realm: SOULSTREAM stream (no MaxAge), personas KV, objects bucket. Setup is a documented script.	Multi-realm tooling, soulctl, clustering.
Record	Full spec: Nats-Msg-Id as op ID, Ss-Author/Ss-Parents/Ss-Type/Ss-Version headers, pure-data payloads, dedup. Library populates parents.	Ss-Sig + canonical-record signing (spec'd, unimplemented).
Ordering	Materialise by stream sequence (JetStream's free total order). DAG recorded, not consulted.	Eg-walker merge. Consequence: rare concurrent ops may render in different order than a future CRDT would choose — acceptable for conversation, revisit with documents.
Personas	Registry KV + transport-scoped credentials; kind as metadata; multiple creds per persona.	Hard-scoping, auth callout, signing keys, delegation tooling.
Topics	topic.announce + initial baseline as first ops message (inline only — MVP state never exceeds the threshold), turn.post, comment.add, attachment.add; lifecycle proposed → active → closed, manual transitions; sub-topics (free — just subject depth).	edit, angle.introduce, comment.reply/resolve, attachment.remove, dormant automation, re-baselining (rollup), manifest baselines, archived. Full logs are replayed — MVP topics are short.
Attachments	Object store put/get, attachment.add with name + digest.	Encryption, lifecycle cleanup.
Mentions	@name parse → mentions array → mention.notify publish; personas subscribe to their own subject.	Digests, presence-aware deferral.
Discovery	Library-local topic projection built by replaying announce + life (cheap, no compaction yet).	Steward persona.
Library	Go (decided 2026-07-11), Layer 1: record construction, publish/replay, materialisation, mentions, projection. Single-binary CLI head and MCP adapter fall out of the same codebase.	TypeScript as impl #2, spec test-suite extraction (tests exist, but portability suite comes with impl #2).

Capability	Minimal slice	Explicitly not yet
Heads	A minimal CLI/TUI head for the human; an MCP adapter so AI personas can participate immediately (one persona’s credentials per session).	WebSocket gateway, browser head, bridges.

MVP definition of done: the dogfood project runs for two weeks; a human and two agents have announced topics, held threaded conversations with mentions and attachments, and closed topics — with no component in the deployment other than NATS, the library, the CLI head, and the MCP adapter.

Day-2 — next, in rough order

1. **Re-baselining (rollup) + manifest baselines + archived** — when replay gets slow or state outgrows the inline threshold. *Gate: signing + archivist decision first (one-way doors).*
2. **Signing** (sig, signing keys, TOFU pinning) — before or with #1.
3. **Steward persona** — when topic count makes library-local projection noisy; also the first real test of “service personas are just participants.”
4. **Work ladder, rungs 1-2** — versioned artefacts and agent work items (see *The work ladder* below).
5. **Eg-walker merge** — gated by work-ladder rung 3 (live co-editing), not before.
6. **Remaining vocabulary** — edit, angle.introduce, comment.reply/resolve, attachment.remove, dormant automation.
7. **Memory convention** (soulstream.svc.memory, citation grading) + first archivist if the realm’s history matters.
8. **Sealed topics** — day-2 by explicit decision; the crypto is the single biggest build item and the dogfood scenario doesn’t need it.
9. **WebSocket/browser head, presence.**

Later

MLS upgrade for sealed topics; bridges (Slack/email); sandbox runtime and its coordination vocabulary; second library language + extracted spec test-suite; soulctl; multi-realm operations.

The work ladder

“Documents/workloads” resolved (2026-07-11) as *all* of: versioned artefacts, agent work items, live co-editing, executable workloads, sandboxes. That is a destination, not a work item — five rungs, each with its own gate, each usable without the next:

Rung	What	New machinery	Gate
1. Versioned artefacts	Document = topic-anchored attachment, revised whole-file (attachment.add anchored to predecessor).	None — existing ops.	Day-2, immediately useful in dogfood.

Rung	What	New machinery	Gate
2. Agent work items	A work-tracking vocabulary (work.open, work.claim, work.done, ...) over ordinary topic op-logs — tasks are conversations with status.	Vocabulary only (additive).	Day-2; design doc needed (07).
3. Live co-editing	Character/block-level ops on shared documents.	Eg-walker lands here. The single biggest library build.	When rung-1 whole-file versioning demonstrably chafes — not before.
4. Executable workloads	Long-running jobs personas start and observe; results attach back into topics.	Execution vocabulary + a runner persona (ordinary credentials — a runner is a participant that does things).	Needs rung 2 (work items are how jobs are asked for).
5. Sandboxes	Shared execution/editing environments with filesystems and processes.	A runtime, outside the substrate; topics carry only its coordination.	Last; design against a working rung-4, per 00-vision.md non-goals.

The ladder's discipline: no rung may be started while the previous rung is undesigned, and rung 3's cost (eg-walker) is paid only when rung 1's limits are felt in real use, not anticipated.